Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут ім. Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЛАБОРАТОРНА РОБОТА № 2

з курсу «Організація баз даних»

Виконали:
Студенти ІІ курсу ФІОТ
Групи ІО-45
Бондарчук М.О.
Романенко І.Є.

Перевірив:
Русінов Володимир Володимирович

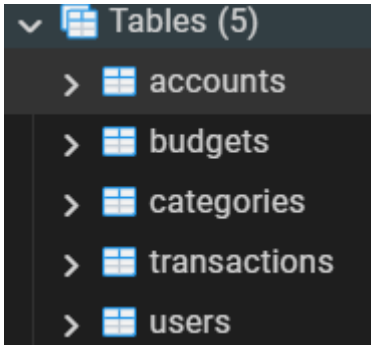
Київ 2026

Мета: використати ER-діаграму з першої роботи для виконання ключових етапів роботи №2

Огляд схеми бази даних

Розроблена база даних призначена для функціонування вебзастосунку,

де користувач заносить свої дані про предмети що там зберігаються і отримує інформацію щодо них за запитом. Схема складається з п'яти таблиць: accounts, budgets, categories, transactions, users.



Опис таблиць

1) **Таблиця accounts**

```
SELECT * FROM accounts;
```

	account_id [PK] integer	user_id integer	account_name character varying (100)	balance numeric (12,2)	currency character varying (10)
1	1	1	Монобанк Картка	15000.00	UAH
2	2	1	Готівка	2000.00	UAH
3	3	2	ПриватБанк	8500.50	UAH

2) **Таблиця budgets**

```
SELECT * FROM budgets;
```

	budget_id [PK] integer	user_id integer	category_id integer	limit_amount numeric (12,2)	start_date date	end_date date
1	1	1	2	8000.00	2023-11-01	2023-11-30
2	2	1	3	1000.00	2023-11-01	2023-11-30
3	3	2	2	5000.00	2023-11-01	2023-11-30

3) **Таблиця categories**

```
SELECT * FROM categories;
```

Data Output Messages Notifications

	category_id [PK] integer	category_name character varying (100)	type character varying (50)
1	1	Зарплата	Income
2	2	Продукти	Expense
3	3	Транспорт	Expense

4) **Таблиця transactions**

```
SELECT * FROM transactions;
```

ata Output Messages Notifications

Showing rows:

transaction_id [PK] integer	account_id integer	category_id integer	amount numeric (12,2)	transaction_date timestamp without time zone	description text
1	1	1	25000.00	2026-03-02 22:19:34.931427	Аванс за жовте...
2	1	2	1200.50	2026-03-02 22:19:34.931427	Сільпо
3	2	3	50.00	2026-03-02 22:19:34.931427	Маршрутка
4	3	2	850.00	2026-03-02 22:19:34.931427	АТБ

5) **Таблиця users**

```
SELECT * FROM users;
```

Showing rows: 1 to 3

user_id [PK] integer	name_surname character varying (100)	email character varying (100)	mobile_number character varying (20)	password_hash character varying (255)	registration_date date
1	Іван Петренко	ivan@example.com	+380501234567	hash123	2023-10-01
2	Олена Коваленко	olena@example.com	+380671234567	hash456	2023-10-02
3	Максим Бойко	maks@example.com	+380631234567	hash789	2023-10-03

Використані обмеження:

-PRIMARY KEY (Первинний ключ): Кожна таблиця у нас має один первинний ключ. Ми використали його для ідентифікаторів (user_id, category_id, account_id, budget_id, transaction_id). У PostgreSQL використання PRIMARY KEY

автоматично означає UNIQUE NOT NULL (унікальне і не порожнє значення).

-FOREIGN KEY / REFERENCES (Зовнішній ключ): Використовується для зв'язку між таблицями та забезпечення цілісності посилань. Наприклад, у таблиці транзакцій ми вказали account_id INTEGER REFERENCES accounts(account_id), що гарантує: транзакцію неможливо створити для рахунку, якого не існує в системі.

-NOT NULL: Це обмеження позначає обов'язкові стовпці, щоб запобігти пропуску важливих даних. Ми застосували його до критичних полів: імені користувача, пароля, назви рахунку, суми транзакції тощо. Наприклад: amount NUMERIC(12, 2) NOT NULL.

-UNIQUE (Унікальність): Використовується, якщо значення в стовпці не може повторюватися. Ми застосували його до пошти користувача: email VARCHAR(100) UNIQUE NOT NULL. Це гарантує, що не можна створити два різні акаунти на одну електронну адресу.

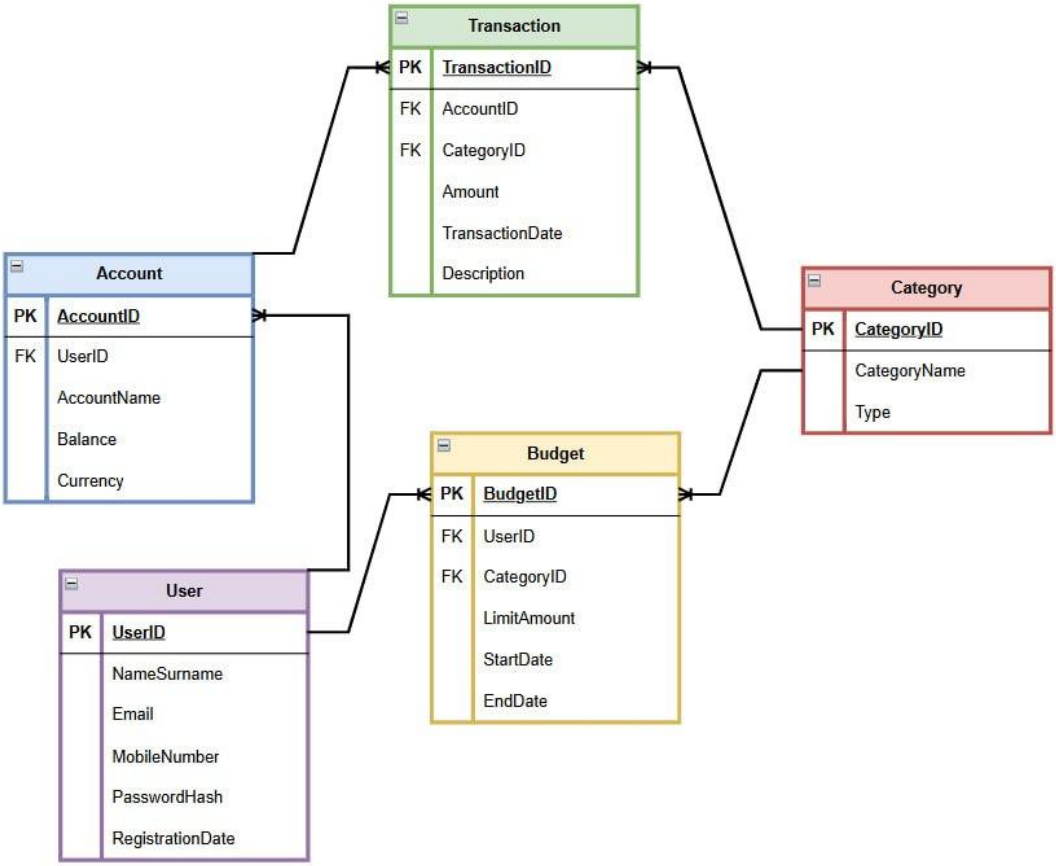
-CHECK (Перевірка умов): Використовується для застосування простих бізнес-правил безпосередньо в базі даних. Ми застосували його у двох місцях:

- У таблиці categories: CHECK (type IN ('Income', 'Expense')). Це гарантує, що категорія може бути лише доходом або витратою, і нічим іншим.
- У таблиці budgets: CHECK (limit_amount > 0). Це правило гарантує, що ліміт бюджету завжди має бути додатним числом.

-DEFAULT (Значення за замовчуванням): Використовується для стовпців, де значення за замовчуванням має сенс, якщо користувач його не надав.

- balance NUMERIC(12, 2) DEFAULT 0.00 (при створенні нового рахунку на ньому 0 грошей).
- registration_date DATE DEFAULT CURRENT_DATE (автоматично підставляє сьогоднішню дату при реєстрації).

Використана ER-діаграма



SQL-скрипт створення та наповнення

```
-- 1. Створення таблиці Users (Користувачі)
CREATE TABLE users (
   user_id SERIAL PRIMARY KEY,
   name_surname VARCHAR(100) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL,
   mobile_number VARCHAR(20),
   password_hash VARCHAR(255) NOT NULL,
   registration_date DATE DEFAULT CURRENT_DATE
);

-- 2. Створення таблиці Categories (Категорії)
CREATE TABLE categories (
   category_id SERIAL PRIMARY KEY,
   category_name VARCHAR(100) NOT NULL,
   type VARCHAR(50) NOT NULL CHECK (type IN ('Income', 'Expense'))
);


-- 3. Створення таблиці Accounts (Рахунки)
CREATE TABLE accounts (
   account_id SERIAL PRIMARY KEY,
   user_id INTEGER REFERENCES users(user_id),
   account_name VARCHAR(100) NOT NULL,
   balance NUMERIC(12, 2) DEFAULT 0.00,
   currency VARCHAR(10) NOT NULL
);


-- 4. Створення таблиці Budgets (Бюджети)
CREATE TABLE budgets (
   budget_id SERIAL PRIMARY KEY,
   user_id INTEGER REFERENCES users(user_id),
   category_id INTEGER REFERENCES categories(category_id),
   limit_amount NUMERIC(12, 2) NOT NULL CHECK (limit_amount > 0),
   start_date DATE NOT NULL,
   end_date DATE NOT NULL
);

-- 5. Створення таблиці Transactions (Транзакції)
CREATE TABLE transactions (
   transaction_id SERIAL PRIMARY KEY,
   account_id INTEGER REFERENCES accounts(account_id),
   category_id INTEGER REFERENCES categories(category_id),
   amount NUMERIC(12, 2) NOT NULL,
   transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   description TEXT
);
```

Вставка тестових даних (INSERT):

```
-- Додаємо 3 користувачів
INSERT INTO users (name_surname, email, mobile_number, password_hash,
registration_date) VALUES
('Іван Петренко', 'ivan@example.com', '+380501234567', 'hash123', '2023-
10-01'),
('Олена Коваленко', 'olena@example.com', '+380671234567', 'hash456',
'2023-10-02'),
('Максим Бойко', 'maks@example.com', '+380631234567', 'hash789', '2023-
10-03');

-- Додаємо 3 категорії
INSERT INTO categories (category_name, type) VALUES
('Зарплата', 'Income'),
('Продукти', 'Expense'),
('Транспорт', 'Expense');

-- Додаємо 3 рахунки
INSERT INTO accounts (user_id, account_name, balance, currency)
VALUES
(1, 'Монобанк Картка', 15000.00, 'UAH'),
(1, 'Готівка', 2000.00, 'UAH'),
(2, 'ПриватБанк', 8500.50, 'UAH');

-- Додаємо 3 бюджети
INSERT INTO budgets (user_id, category_id, limit_amount, start_date,
end_date) VALUES
(1, 2, 8000.00, '2023-11-01', '2023-11-30'), -- Іван планує витратити не
більше 8000 на продукти
(1, 3, 1000.00, '2023-11-01', '2023-11-30'), -- Іван планує витратити не
більше 1000 на транспорт
(2, 2, 5000.00, '2023-11-01', '2023-11-30'); -- Олена планує витратити не
більше 5000 на продукти

-- Додаємо 4 транзакції
INSERT INTO transactions (account_id, category_id, amount, description)
VALUES
(1, 1, 25000.00, 'Аванс за жовтень'),
(1, 2, 1200.50, 'Сільпо'),
(2, 3, 50.00, 'Маршрутка'),
(3, 2, 850.00, 'АТБ');
```

Висновок:

Результатом роботи стала валідна реляційна база даних у PostgreSQL, розгорнута шляхом декомпозиції ER-моделі. Впроваджена система ключів гарантує посилальну цілісність, а використання ENUM дозволило мінімізувати логічні помилки на рівні схеми. Тестування довело надійність та готовність структури до експлуатації.